Centre number	5	1	3	3	7	Candidate number	1	0	6	3
Surname	Mahajan									
Forename(s)	Vaibhav									
Candidate signature										
Programming Language	**Python (7517/1D)**									

For Examiner's Use
Examiner's Initials

Question	Mark
1	
2	
3	
4	
5	
6	
7	
8	
9	-6
10	
11	
12	
13	
14	
15	
TOTAL	94

A-LEVEL COMPUTER SCIENCE

Paper 1

Friday 28 May 2021
Afternoon
Time allowed: 2 hours 30 minutes

Instructions
- This is the Electronic Answer Document (EAD). Answer **all** questions by entering your answers into this document on screen. You **must save** this document at regular intervals.
- Before the examination begins, type the information needed in the boxes **at the top of this page**.
- Before the examination begins, type the information needed in the boxes **in the footers** (page 2 onwards) of this EAD.

During the examination
- You may print pages of your EAD. A print monitor will collect and deliver your print-out to you. You must **not** collect your own print-out.

Exceptions
- If you experience difficulty inserting screen shots into your EAD then you may print these separately and attach them to the back of the EAD with a reference in the correct place in the EAD. Ensure that your **Centre Number**, **Candidate Name** and **Candidate Number** are on each sheet.

At the end of the examination
- Save for the last time and print your EAD **on one side only** (not double-sided). A print monitor will collect and deliver your print-out to you. Check that your details are in the footers of every page. Write them in if they are not.
- Enter your signature on the front cover.
- Staple or tie all pages together in the top left-hand corner of the EAD.
- Hand in **all** pages of the EAD to the Invigilator.

Warning
- No extra time is allowed for printing and collating.
- It may not be possible to credit an answer if your:
  – details are not printed on every page as instructed above
  – screen captures are not legible to the Examiner.

Answer **all** questions.

You **must save** this document at regular intervals or you may lose your work.

Section A

Leave blank

Question 01

0 1 . 0

3	5	1	6	4	8
3	1	5	4	6	8
1	3	4	5	6	8

3

Rooted

Leave blank

Question 02

0 2 . 1 A tree where the nodes have at most two children; a tree is an undirected graph with no cycles.

0 2 . 2 E,I,H,C,Y,B,Q

0 2 . 3

C	P	0	1	2	3	O
0	0	0				
0	-1					C
	0	4				
	1		1			
1	0					I
	1		3			
	2			2		
2	1					E
3	0					H
4	-1					B
	0	6				
	1		5			
5	0					Y
6	-1					Q

0 2 . 4 A subroutine that calls itself

0 2 . 5 The case when the subroutine returns without calling itself; the case where the stack can unwind.

0 2 . 6 The return address, local variables and arguments to the subroutine

Centre Number:	Candidate Name:	Candidate Number:

Personally

15

Centre Number:					Candidate Name:	Candidate Number:			

Leave blank

Question 03

0	3	.	0	The breaking down of a program into callable subroutines that can be re-used and are called by invoking the subroutine.	

3

Composite subroutines 1. task =2

Leave blank

Question 04

0	4	.	0	The key of the item is hashed using the hash function, which generates the index for where the item should be stored. The record is stored along with its key. If the index is already occupied, a collision has occurred because the hash function generated the same index for two different records. In this case, a rehashing algorithm is used to find another empty slot, which is where the record is stored. The rehashing algorithm could be simply finding the nearest empty slot, or it compute the index using another hash function.	

5

Simpler

Leave blank

Question 05

0	5	.	1	No need for brackets, since there is no ambiguity. Also, it is easier for a computer to evaluate than infix notation.	
0	5	.	2	The numbers are pushed onto the stack, and when an operator is encountered, the last two numbers on the stack are popped, the result evaluated, and then the result is pushed back onto the stack. At the end of the expression, only a single value should remain in the stack.	

6

Pop

Leave blank

Question 06

0	6	.	1		

S2	a	S5
S2	b	S4
S0	b	S2
S5	b	S2

0	6	.	2	a(ba)*|b(ab)*	

5

Centre Number:	Candidate Name:	Candidate Number:

Centre Number:					Candidate Name:	Candidate Number:			

Section B

Leave blank

Question 07

0 7 . 1

```
n = int(input("Enter a number..."))
i = 0
num_counted = 0
while num_counted < n:
    i+=1
    digits = []
    str_i = str(i) # String representation of i
    for str_digit in str_i: # Add each digit to list of digits
        digits.append(int(str_digit))
    sum_digits = sum(digits) # sum of the digits
    if i % sum_digits == 0:
        num_counted += 1
print(f"Harshad number {n} is {i}")
```

0 7 . 2

```
Enter a number...600
Harshad number 600 is 3102
```

13

Section C

Leave blank

Question 08

0	**8**	**.**	**1**	One of the co-ordinates will be the same; the x,y or z co-ordinate will match for both of them.
0	**8**	**.**	**2**	
0	**8**	**.**	**3**	It would need to take in 3 parameters per tile (6 in total) rather than two indices, and it would need to calculate the index from the coordinates to access the tiles.

5

Leave blank

Question 09

0	**9**	**.**	**1**	Because sub-classes have different values for it, and so in order for them to be able to change it, it needs to be protected
0	**9**	**.**	**2**	On half a turn
0	**9**	**.**	**3**	Where subclasses can inherit from parent classes, and override their behaviour while still being an 'is-a' relationship.

Interface

4

Centre Number:	Candidate Name:	Candidate Number:

					Leave blank
Question 10					
1	**0**	**.**	**1**	Inputting a non-numeric value when the program is expecting a number.	
1	**0**	**.**	**2**	`CheckMoveCommandFormat`	
					2

					Leave blank
Question 11					
1	**1**	**.**	**0**	The maximum of the difference in the x coordinates, y coordinates and z coordinates, ensuring that the distances are positive through the use of the absolute value function (abs).	
					2

Centre Number:					Candidate Name:	Candidate Number:			

Section D

Leave blank

Question 12

1 2 . 1

```
def DestroyPiecesAndCountVPs(self):
  BaronDestroyed = False
  Player1VPs = 0
  Player2VPs = 0
  ListOfTilesContainingDestroyedPieces = []
  for T in self._Tiles:
    if T.GetPieceInTile() is not None:
      ListOfNeighbours = T.GetNeighbours()
      NoOfConnections = 0
      for N in ListOfNeighbours:
        if N.GetPieceInTile() is not None:
          NoOfConnections += 1
      ThePiece = T.GetPieceInTile()
      if NoOfConnections >=
ThePiece.GetConnectionsNeededToDestroy():
        ThePiece.DestroyPiece()
        if ThePiece.GetPieceType().upper() == "B":
          BaronDestroyed = True
        ListOfTilesContainingDestroyedPieces.append(T)
        if ThePiece.GetBelongsToPlayer1():
          Player2VPs += ThePiece.GetVPs()
        else:
          Player1VPs += ThePiece.GetVPs()
      else:
        if ThePiece.GetPieceType().upper() == "L":
          if ThePiece.GetBelongsToPlayer1():
            Player1VPs += 1
          else:
            Player2VPs += 1
  for T in ListOfTilesContainingDestroyedPieces:
    T.SetPiece(None)
  return BaronDestroyed, Player1VPs, Player2VPs
```

Centre Number:	Candidate Name:	Candidate Number:

1 2 . 2

```
Enter your choice: 2
Enter the name of the file to load: game1.txt
Player One current state - VPs: 0   Pieces in supply: 2   Lumber: 5   Fuel: 5
Player Two current state - VPs: 0   Pieces in supply: 2   Lumber: 5   Fuel: 5


  __    __    __
 /#B\__/#b\__/~l\__
 \__/~L\__/  \__/  \
 /  \__/~ \__/  \__/
 \__/  \__/  \__/  \
 /  \__/#l\__/# \__/
 \__/#L\__/~ \__/~S\
    \__/  \__/  \__/

Player One state your three commands, pressing enter after each one.
Enter command:
Enter command:
Enter command:
Invalid command
Invalid command
Invalid command
Player One current state - VPs: 1   Pieces in supply: 2   Lumber: 5   Fuel: 5
Player Two current state - VPs: 5   Pieces in supply: 2   Lumber: 5   Fuel: 5
Press Enter to continue...


  __    __    __
 /#B\__/#b\__/~l\__
 \__/~ \__/  \__/  \
 /  \__/~ \__/  \__/
 \__/  \__/  \__/  \
 /  \__/#l\__/# \__/
 \__/#L\__/~ \__/~S\
    \__/  \__/  \__/

Player Two state your three commands, pressing enter after each one.
Enter command:
Enter command:
Enter command:
Invalid command
Invalid command
Invalid command
Player One current state - VPs: 2   Pieces in supply: 2   Lumber: 5   Fuel: 5
Player Two current state - VPs: 7   Pieces in supply: 2   Lumber: 5   Fuel: 5
Press Enter to continue...


  __    __    __
 /#B\__/#b\__/~l\__
 \__/~ \__/  \__/  \
 /  \__/~ \__/  \__/
 \__/  \__/  \__/  \
 /  \__/#l\__/# \__/
 \__/#L\__/~ \__/~S\
    \__/  \__/  \__/
```

6

Leave blank

Question 13

1 3 . 1

```
def AddPiece(self, BelongsToPlayer1, TypeOfPiece, Location):
  if TypeOfPiece == "Baron":
    NewPiece = BaronPiece(BelongsToPlayer1)
  elif TypeOfPiece == "LESS":
```

Centre Number:	Candidate Name:	Candidate Number:

```
      NewPiece = LESSPiece(BelongsToPlayer1)
    elif TypeOfPiece == "PBDS":
      NewPiece = PBDSPiece(BelongsToPlayer1)
    elif TypeOfPiece == "Ranger":
      NewPiece = RangerPiece(BelongsToPlayer1)
    else:
      NewPiece = Piece(BelongsToPlayer1)
    self._Pieces.append(NewPiece)
    self._Tiles[Location].SetPiece(NewPiece)


class RangerPiece(Piece):
  def __init__(self, Player1):
    super(RangerPiece,self).__init__(Player1)
    self._PieceType = "R"

  def CheckMoveIsValid(self, DistanceBetweenTiles, StartTerrain,
EndTerrain):
    normal = super().CheckMoveIsValid(DistanceBetweenTiles,
StartTerrain, EndTerrain)
    if normal != -1:
      return normal
    elif StartTerrain == '#' and EndTerrain == '#':
      return 1
    else:
      return -1
```

Fuel cost of move

Centre Number:	Candidate Name:	Candidate Number:

1 3 . 2

```
1. Default game
2. Load game
Q. Quit

Enter your choice: 1
Player One current state - VPs: 0   Pieces in supply: 5   Lumber: 10   Fuel: 10
Player Two current state - VPs: 1   Pieces in supply: 5   Lumber: 10   Fuel: 10

  __    __    __    __
 / B\__/# \__/# \__/  \__
 \__/~ \__/~ \__/  \__/  \
 / R\__/~ \__/  \__/# \__/
 \__/# \__/  \__/  \__/  \
 /  \__/  \__/# \__/# \__/
 \__/# \__/# \__/~ \__/~s\
 /~ \__/~ \__/~ \__/  \__/
 \__/# \__/  \__/# \__/ b\
    \__/  \__/  \__/  \__/

Player One state your three commands, pressing enter after each one.
Enter command: move 8 12
Enter command: move 12 2
Enter command: move 2 3
Command executed
Command executed
That move can't be done
Player One current state - VPs: 0   Pieces in supply: 5   Lumber: 10   Fuel: 8
Player Two current state - VPs: 1   Pieces in supply: 5   Lumber: 10   Fuel: 10
Press Enter to continue...

  __    __    __    __
 / B\__/# \__/#R\__/  \__
 \__/~ \__/~ \__/  \__/  \
 /  \__/~ \__/  \__/# \__/
 \__/# \__/  \__/  \__/  \
 /  \__/  \__/# \__/# \__/
 \__/# \__/# \__/~ \__/~s\
 /~ \__/~ \__/~ \__/  \__/
 \__/# \__/  \__/# \__/ b\
    \__/  \__/  \__/  \__/
```

8

Leave blank

Question 14

1 4 . 1

```python
def CheckCommandIsValid(Items):
  if len(Items) > 0:
    if Items[0] == "move":
      return CheckMoveCommandFormat(Items)
    elif Items[0] in ["dig", "saw", "spawn"]:
      return CheckStandardCommandFormat(Items)
    elif Items[0] == "upgrade":
      return CheckUpgradeCommandFormat(Items)
    elif Items[0] == "burn": #Q14
      return True
  return False


  def ExecuteCommand(self, Items, FuelAvailable, LumberAvailable,
```

Centre Number:	Candidate Name:	Candidate Number:

```
PiecesInSupply):
    FuelChange = 0
    LumberChange = 0
    SupplyChange = 0
    if Items[0] == "move":
      FuelCost = self.__ExecuteMoveCommand(Items, FuelAvailable)
      if FuelCost < 0:
        return "That move can't be done", FuelChange,
LumberChange, SupplyChange
      FuelChange = -FuelCost
# Q14
    elif Items[0] == "burn":
      if LumberAvailable <= 0:
        return "Cannot burn lumber",FuelChange, LumberChange,
SupplyChange
      else:
        randnum = random.randint(1,LumberAvailable)
        LumberChange = -randnum
        FuelChange = randnum
#/Q14
    elif Items[0] in ["saw", "dig"]:
      Success, FuelChange, LumberChange =
self.__ExecuteCommandInTile(Items)
      if not Success:
        return "Couldn't do that", FuelChange, LumberChange,
SupplyChange
    elif Items[0] == "spawn":
      LumberCost = self.__ExecuteSpawnCommand(Items,
LumberAvailable, PiecesInSupply)
      if LumberCost < 0:
        return "Spawning did not occur", FuelChange, LumberChange,
SupplyChange
      LumberChange = -LumberCost
      SupplyChange = 1
    elif Items[0] == "upgrade":
      LumberCost = self.__ExecuteUpgradeCommand(Items,
LumberAvailable)
      if LumberCost < 0:
        return "Upgrade not possible", FuelChange, LumberChange,
SupplyChange
      LumberChange = -LumberCost
    return "Command executed", FuelChange, LumberChange,
SupplyChange
```

Centre Number:	Candidate Name:	Candidate Number:

1 4 . 2

```
Enter your choice: 1
Player One current state - VPs: 0   Pieces in supply: 5   Lumber: 10   Fuel: 10
Player Two current state - VPs: 1   Pieces in supply: 5   Lumber: 10   Fuel: 10

 __    __    __    __
/ B\__/# \__/# \__/  \__
\__/~ \__/~ \__/  \__/  \
/ R\__/~ \__/  \__/# \__/
\__/# \__/  \__/  \__/  \
/  \__/  \__/# \__/# \__/
\__/# \__/# \__/~ \__/~s\
/~ \__/~ \__/~ \__/  \__/
\__/# \__/  \__/# \__/ b\
   \__/  \__/  \__/  \__/

Player One state your three commands, pressing enter after each one.
Enter command: burn
Enter command:
Enter command:
Command executed
Invalid command
Invalid command
Player One current state - VPs: 0   Pieces in supply: 5   Lumber: 6   Fuel: 14
Player Two current state - VPs: 1   Pieces in supply: 5   Lumber: 10   Fuel: 10
Press Enter to continue...
```

9

Leave blank

Question 15

1 5 . 1

```
def GetPieceTypeInTile(self, ID):
  ThePiece = self._Tiles[ID].GetPieceInTile()
  if ThePiece is None:
    return " "
  elif self.GetFogOfWar(ID):
    return " " # Hidden
  else:
    return ThePiece.GetPieceType()

def GetFogOfWar(self,ID):
  for Tile in self._Tiles: # Effectively iterating through all the
pieces
    piece = Tile.GetPieceInTile()
    if piece is not None:
      if piece.GetBelongsToPlayer1() == self._Player1Turn:
        Distance = self._Tiles[ID].GetDistanceToTileT(Tile) #
Distance between the piece and tile at ID
        if Distance <= 2:
          return False # Not hidden
  return True
```

Centre Number:	Candidate Name:	Candidate Number:

1 5 . 2

```
Enter your choice: 2
Enter the name of the file to load: game1.txt
Player One current state - VPs: 0   Pieces in supply: 2   Lumber: 5   Fuel: 5
Player Two current state - VPs: 0   Pieces in supply: 2   Lumber: 5   Fuel: 5

  __    __    __
 /#B\__/#b\__/~ \__
 \__/~L\__/  \__/  \
 /  \__/~ \__/  \__/
 \__/  \__/  \__/  \
 /  \__/#l\__/# \__/
 \__/#L\__/~ \__/~S\
    \__/  \__/  \__/

Player One state your three commands, pressing enter after each one.
Enter command:
Enter command:
Enter command:
Invalid command
Invalid command
Invalid command
Player One current state - VPs: 1   Pieces in supply: 2   Lumber: 5   Fuel: 5
Player Two current state - VPs: 5   Pieces in supply: 2   Lumber: 5   Fuel: 5
Press Enter to continue...

  __    __    __
 /#B\__/#b\__/~l\__
 \__/~ \__/  \__/  \
 /  \__/~ \__/  \__/
 \__/  \__/  \__/  \
 /  \__/#l\__/# \__/
 \__/#L\__/~ \__/~ \
    \__/  \__/  \__/
```

14

Centre Number:	Candidate Name:	Candidate Number:

If you have made any separate screen captures, put a reference in the correct place in this Electronic Answer Document.

Ensure that your Centre Number, Candidate Name and Candidate Number are on each sheet.

Attach them to the back of this document.

NB: The AQA logo is provided for use only in the Electronic Answer Document for the 2021 A-level Computer Science Paper 1 (7517/1) examination and must not be used for any other purpose.

Copyright © 2021 AQA and its licensors. All rights reserved.

Centre Number:					Candidate Name:	Candidate Number:			